

Monte Carlo Methods and Direct Policy Search

Reinforcement Learning

Christian Igel

Department of Computer Science

🐦 @christian_igel



Outline

Policy Improvement Theorem

Monte Carlo Methods

Direct Policy Search

- Cross-entropy Method

- Covariance Matrix Adaption Evolution Strategy

- Limitations

Outline

Policy Improvement Theorem

Monte Carlo Methods

Direct Policy Search

- Cross-entropy Method

- Covariance Matrix Adaption Evolution Strategy

- Limitations

Notation

- Discrete time t , states S , actions A
- State-action-reward sequence $s_0, a_0, r_1, \dots$, with first time arriving in a terminal state in episodic tasks denoted by T (and for $t > T$: $r_t = 0$ and $s_t = s_T$), start state distribution p_{start}
- $P_{ss'}^a = \Pr\{s_{t+1} = s' \mid s_t = s, a_t = a\}$ and $R_{ss'}^a = \mathbb{E}\{r_{t+1} \mid s_t = s, s_{t+1} = s', a_t = a\}$
- $R_s^a = \mathbb{E}\{r_{t+1} \mid s_t = s, a_t = a\} = \sum_{s'} P_{ss'}^a R_{ss'}^a$

Policy improvement theorem: Idea

- Recall state-value function for a state s given policy π

$$V^\pi(s) = \mathbb{E}^\pi \{R_t \mid s_t = s\} = \mathbb{E}^\pi \left\{ \sum_{k=0}^{\infty} \gamma^k r_{t+k+1} \mid s_t = s \right\}$$

and suppose we have computed V^π for a deterministic policy π .

The value of doing a in s is:

$$\begin{aligned} Q^\pi(s, a) &= \mathbb{E}^\pi \{r_{t+1} + \gamma V^\pi(s_{t+1}) \mid s_t = s, a_t = a\} \\ &= \sum_{s'} P_{ss'}^a [R_{ss'}^a + \gamma V^\pi(s')] \end{aligned}$$

- For a given state s , would it be better to do an action $a \neq \pi(s)$?
- It is better to switch to action a for state s if and only if:

$$Q^\pi(s, a) > V^\pi(s)$$

Policy improvement theorem

Given two policies $\pi, \pi' : S \rightarrow A$. If for *all* states $s \in S$

$$Q^\pi(s, \pi'(s)) \geq V^\pi(s)$$

and for *some* states $s \in S$

$$Q^\pi(s, \pi'(s)) > V^\pi(s)$$

then for *all* states $s \in S$

$$V^{\pi'}(s) \geq V^\pi(s)$$

and for *some* states $s \in S$

$$V^{\pi'}(s) > V^\pi(s) \text{ .}$$

Policy improvement theorem proof

$$\begin{aligned} V^\pi(s) &\leq Q^\pi(s, \pi'(s)) \\ &= \mathbb{E}^\pi \{r_{t+1} + \gamma V^\pi(s_{t+1}) \mid s_t = s, a_t = \pi'(s)\} \\ &= \mathbb{E}^{\pi'} \{r_{t+1} + \gamma V^\pi(s_{t+1}) \mid s_t = s\} \\ &\leq \mathbb{E}^{\pi'} \{r_{t+1} + \gamma Q^\pi(s_{t+1}, \pi'(s_{t+1})) \mid s_t = s\} \\ &= \mathbb{E}^{\pi'} \{r_{t+1} + \gamma \mathbb{E} \{r_{t+2} + \gamma V^\pi(s_{t+2}) \mid s_{t+1}, a_{t+1} = \pi'(s_{t+1})\} \mid s_t = s\} \\ &= \mathbb{E}^{\pi'} \{r_{t+1} + \gamma r_{t+2} + \gamma^2 V^\pi(s_{t+2}) \mid s_t = s\} \\ &\dots \\ &\leq \mathbb{E}^{\pi'} \{r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \gamma^3 V^\pi(s_{t+3}) \mid s_t = s\} \\ &\dots \\ &\leq \mathbb{E}^{\pi'} \{r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \gamma^3 r_{t+4} + \dots \mid s_t = s\} \\ &= V^{\pi'}(s) \quad (\text{for all } s \in S) \end{aligned}$$

Policy improvement

Change policy accordingly for all states to get a new policy π' that is **greedy** with respect to V^π :

$$\begin{aligned}\pi'(s) &= \operatorname{argmax}_a Q^\pi(s, a) \\ &= \operatorname{argmax}_a \sum_{s'} P_{ss'}^a [R_{ss'}^a + \gamma V^\pi(s')]\end{aligned}$$

Then $\forall s \in S : V^{\pi'}(s) \geq V^\pi(s)$.

$V^{\pi'} = V^\pi$ implies $\forall s \in S : V^\pi(s) = \max_a \sum_{s'} P_{ss'}^a [R_{ss'}^a + \gamma V^\pi(s')]$.

That's the Bellman optimality equation and $V^\pi = V^{\pi'} = V^*$.



Policy iteration: Idea

Alternate:

1. Policy evaluation
2. Policy improvement

$$\pi_0 \xrightarrow{\text{evaluate}} V^{\pi_0} \xrightarrow{\text{improve}} \pi_1 \xrightarrow{\text{evaluate}} V^{\pi_1} \xrightarrow{\text{improve}} \dots \xrightarrow{\text{improve}} \pi^* \xrightarrow{\text{evaluate}} V^{\pi^*} \xrightarrow{\text{improve}} \pi^*$$

Outline

Policy Improvement Theorem

Monte Carlo Methods

Direct Policy Search

- Cross-entropy Method

- Covariance Matrix Adaption Evolution Strategy

- Limitations

Monte Carlo methods

Crude Monte Carlo methods approximate an integral

$$\mathbb{E}[f(z)] = \int f(z)p(z)\mathrm{d}z$$

by

$$\frac{1}{n} \sum_{i=1}^n f(z_i) \xrightarrow{n \rightarrow \infty} \mathbb{E}_{p(z)}[f(z)] \quad ,$$

where

$$z_i \sim p$$

for $i = 1, \dots, n$.

Monte Carlo methods for RL

- Monte Carlo methods learn from complete sample returns: Integral

$$V^{\pi}(s) = \mathbb{E}^{\pi} \{R_t \mid s_t = s\}$$

is approximated from trajectories sampled following π and passing through s

- Monte Carlo methods learn directly from experience
 - Simulated: No need for a full model
 - Only defined for episodic tasks



Monte Carlo policy iteration

In Monte Carlo policy iteration we combine:

1. Monte Carlo Policy evaluation

- Goal: learn $V^\pi(s)$ or $Q^\pi(s, a)$
- Given: some number of episodes under π which contain s
- Idea: average returns observed after visits to s
 - Every-Visit MC: average returns for every time s is visited in an episode
 - First-visit MC: average returns only for first time s is visited in an episode
 - Both converge asymptotically

2. Policy improvement

Monte Carlo control

Algorithm: Monte Carlo control with exploring starts

Input: arbitrary policy π and state-value function Q

```
1 foreach  $(s, a) \in S \times A$  do
2    $R(s, a) \leftarrow 0$  // accumulated returns
3    $c(s, a) \leftarrow 0$  // counter
4 repeat
5   generate an episode using exploring starts and  $\pi$ 
6   foreach pair  $s, a$  appearing in the episode do
7      $R(s, a) \leftarrow R(s, a) +$  return following first occurrence of  $s, a$ 
8      $c(s, a) \leftarrow c(s, a) + 1$ 
9      $Q(s, a) \leftarrow R(s, a) / c(s, a)$  // policy evaluation
10  foreach state  $s$  appearing in the episode do
11     $\pi(s) \leftarrow \operatorname{argmax}_a Q(s, a)$  // policy improvement
12 until some stopping criterion is met
```

Output: $\pi \approx \pi^*, Q \approx Q^*$

Sutton & Barto example

Assume you have observed the following 8 state-reward sequences:

A, 0, B, 0

B, 1

B, 1

B, 1

B, 1

B, 1

B, 1

B, 0

What is $V(A)$ and $V(B)$?

Outline

Policy Improvement Theorem

Monte Carlo Methods

Direct Policy Search

- Cross-entropy Method

- Covariance Matrix Adaption Evolution Strategy

- Limitations

Direct policy search

RL can be viewed as an optimization problem. Given $\{\pi_{\theta} \mid \theta \in \Theta\}$:

$$\pi^* = \arg \max_{\theta \in \Theta} J(\pi_{\theta}) \quad \text{with} \quad J(\pi_{\theta}) = \mathbb{E}^{\pi_{\theta}}[R]$$

Monte Carlo methods allow to approximate $J(\pi_{\theta})$ by

$$J^{\text{MC}}(\pi_{\theta}) = \frac{1}{n_{\text{MC}}} \sum_{m=1}^{n_{\text{MC}}} \sum_{t=0}^{T_m} \gamma^t r_{t+1}^m \approx \mathbb{E}^{\pi_{\theta}}[R] = J(\pi_{\theta}) \quad ,$$

based on n_{MC} episodes

$$s_0^m, a_0^m, r_1^m, s_1^m, a_1^m, r_2^m, \dots, a_{T_m-1}^m, r_{T_m}^m, s_{T_m}^m$$

following π starting from $s_0^m \sim p_{\text{start}}$ and having length T_m for $m = 1, \dots, n_{\text{MC}}$.

Random search

Iterative random search:

- Start with initial search distribution over Θ .
- In each iteration,
 1. draw candidate policies from the search distribution,
 2. evaluate the policies, and
 3. updates the search distribution based on the better candidates.

The idea is to update the search distribution so that sampling high-quality policies in the next iteration becomes more likely.

Objective function can be non-continuous, multi-modal, noisy, and constrained.

Cross-entropy (CE) method

- Assume $\Theta = \mathbb{R}^n$ and multivariate normal search distribution $\mathcal{N}(\boldsymbol{\theta}, \boldsymbol{C})$ with mean $\boldsymbol{\theta}$ and covariance matrix $\boldsymbol{C} \in \mathbb{R}^{n \times n}$.
Let $\boldsymbol{C}$ be diagonal, $\boldsymbol{C} = \text{diag}(\sigma_1, \sigma_2, \dots, \sigma_n)$, parametrized by $\boldsymbol{\sigma} \in \mathbb{R}^n$.
- In each iteration, we draw λ candidate policies $\boldsymbol{\theta}_l$ and evaluate them by computing $J^{\text{MC}}(\boldsymbol{\theta}_l)$. Let the i -th best of the λ policies be denoted by $\boldsymbol{\theta}_{i:\lambda}$.
- The mean of the distribution is updated to be the mean of the $\mu < \lambda$ best of the λ candidates, and $\boldsymbol{C}$ is updated to be the variance of the selected μ candidates:

$$\begin{aligned}\boldsymbol{\theta} &\leftarrow \frac{1}{\mu} \sum_{i=1}^{\mu} \boldsymbol{\theta}_{i:\lambda} \\ \boldsymbol{\sigma} &\leftarrow \text{Var}[\{\boldsymbol{\theta}_{1:\lambda}, \dots, \boldsymbol{\theta}_{\mu:\lambda}\}]\end{aligned}$$

CE algorithm

Algorithm: Cross-entropy method

Input: $\theta = \theta_{\text{init}}, \sigma = \sigma_{\text{init}}, \mu, \lambda$

Set: ϵ

1 **repeat**

 // sample new policies

2 **for** $l = 1, \dots, \lambda$ **do** $\theta_l \sim \mathcal{N}(\theta, \text{diag}(\sigma_1 + \epsilon, \sigma_2 + \epsilon, \dots))$

 // evaluate new policies

3 **for** $l = 1, \dots, \lambda$ **do** evaluate $J^{\text{MC}}(\theta_l)$

 // update mean

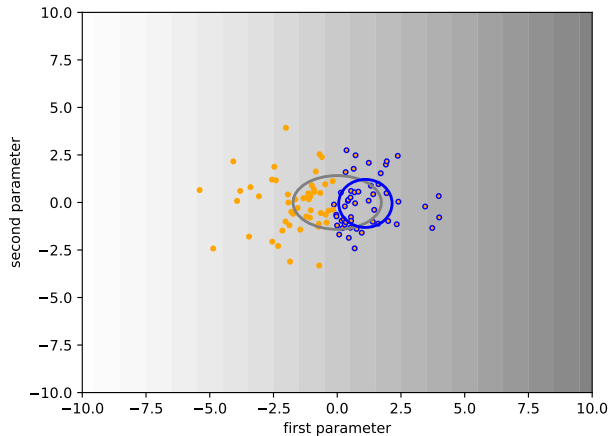
4 $\theta \leftarrow \frac{1}{\mu} \sum_{i=1}^{\mu} \theta_{i:\lambda}$

 // update variance

5 $\sigma \leftarrow \text{Var}[\{\theta_{1:\lambda}, \dots, \theta_{\mu:\lambda}\}]$

6 **until** *some stopping criterion is met*

CE: Conceptual problem

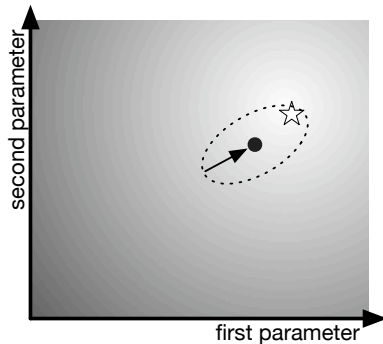


Covariance Matrix Adaption Evolution Strategy (CMA-ES)

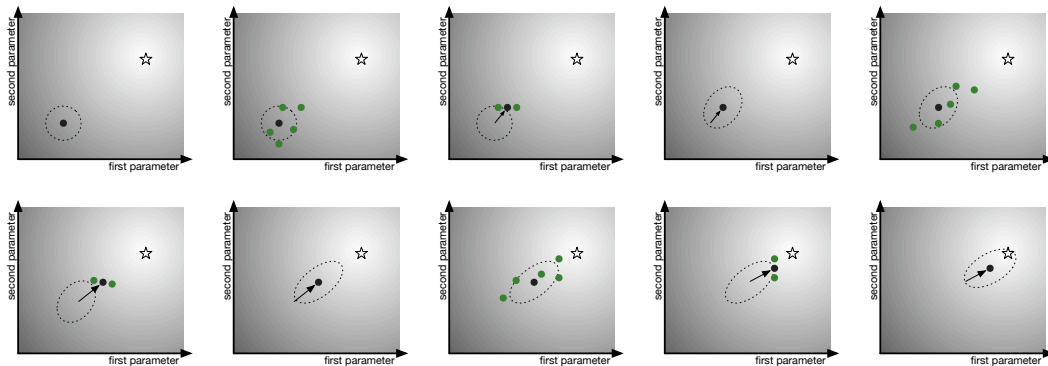
- New candidates are samples from

$$\mathcal{N}(\boldsymbol{\theta}, \sigma^2 \mathbf{C}) =$$


- Covariance matrix $\mathbf{C}$ is updated based on recent *steps*.
- Global step size σ and matrix $\mathbf{C}$ are updated independently, σ can be adjusted on faster timescale.
- Highly efficient, allows for small population sizes



Strategy adaptation in ES cartoon



CMA-ES: Sampling and mean update

- Sampling of new λ candidate policies:

$$\mathbf{y}_l \sim \mathcal{N}(\mathbf{0}, \mathbf{C})$$

$$\boldsymbol{\theta}_l = \boldsymbol{\theta} + \sigma \mathbf{y}_k \text{ .}$$

$$\Rightarrow \boldsymbol{\theta}_l \sim \mathcal{N}(\boldsymbol{\theta}, \sigma^2 \mathbf{C}) =$$


- Mean $\boldsymbol{\theta}$ is updated by weighted sum

$$\mathbf{y}_w \leftarrow \sum_{i=1}^{\mu} w_i \mathbf{y}_{i:\lambda}$$

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \sigma \mathbf{y}_w \text{ .}$$

with

$$\sum_{i=1}^{\mu} w_i = 1 \quad \text{and} \quad w_1 \geq w_2 \geq \dots \geq w_{\mu} \text{ .}$$

Rank-1 covariance matrix update

- Covariance matrix is updated to make recent beneficial steps more likely
- Consider $\mathcal{N}(0, \mathbf{C}' + \epsilon \mathbf{I})$ for $\epsilon \rightarrow 0$. Which $\mathbf{C}'$ makes sampling $\sqrt{\mu_w} \mathbf{y}_w$ most likely? The best choice is the rank-1 matrix $\mathbf{C}' = \mu_w \mathbf{y}_w \mathbf{y}_w^\top$.
- Exponential moving average (evolution path) takes previous steps into account, similar a momentum term in stochastic gradient descent:

$$\mathbf{p}_c \leftarrow (1 - c_c) \mathbf{p}_c + \sqrt{1 - (1 - c_c)^2} \sqrt{\mu_w} \mathbf{y}_w$$

Weights counterbalance change in variance $((1 - c_c)^2 + \sqrt{1 - (1 - c_c)^2}^2 = 1)$.

- This gives

$$\mathbf{p}_c \mathbf{p}_c^\top \leftarrow (1 - \alpha) \mathbf{p}_c \mathbf{p}_c^\top + \alpha \mu_w \mathbf{y}_w \mathbf{y}_w^\top$$

with smoothing factor $\alpha = 1 - (1 - c_c)^2$.

Rank-1 update properties

- Smoothing stabilizes the update allowing for small λ .
- Need to use $\mathbf{p}_c$ instead of the simpler looking $\mathbf{p}_c \mathbf{p}_c^\top$, because sign is important, which is lost in $\mathbf{p}_c \mathbf{p}_c^\top$ ($\mathbf{y}_w \mathbf{y}_w^\top = (-\mathbf{y}_w)(-\mathbf{y}_w)^\top$).
- This covariance update is called rank-1 update, because $\mathbf{C}$ is updated by the matrix $\mathbf{y}_w \mathbf{y}_w^\top$ of rank one.

Rank- μ covariance matrix update

- For large λ and μ , rank- μ updates increases learning speed:

$$\mathbf{C} \leftarrow (1 - c_\mu)\mathbf{C} + c_\mu \sum_{i=1}^{\mu} w_i \mathbf{y}_{i:\lambda} \mathbf{y}_{i:\lambda}^\top ,$$

where for non-zero $w_1, \dots, w_\mu$ the matrix added to $\mathbf{C}$ has rank $\min(\mu, n)$ almost surely, c_μ is a smoothing factor.

- Putting **rank-1** and **rank- μ** update together, we get:

$$\begin{aligned} \mathbf{C} &\leftarrow \mathbf{C} + c_\mu \sum_{i=1}^{\lambda} w_i (\mathbf{y}_{i:\lambda} \mathbf{y}_{i:\lambda}^\top - \mathbf{C}) + c_1 (\mathbf{p}_c \mathbf{p}_c^\top - \mathbf{C}) \\ &= (1 - c_1 - c_\mu) \mathbf{C} + c_\mu \sum_{i=1}^{\mu} w_i \mathbf{y}_{i:\lambda} \mathbf{y}_{i:\lambda}^\top + c_1 \mathbf{p}_c \mathbf{p}_c^\top \end{aligned}$$

For c_1 and c_μ default values can be used, scaling with $O(1/n^2)$ following the degrees of freedom of $\mathbf{C}$.

Weighted mean

- $w_1 = w_2 = \dots = w_\mu = \mu^{-1}$ and $w_{\mu+1} = \dots = w_\lambda = 0$: CE update
- A common choice is

$$w_i = \frac{w'_i}{\sum_{j=1}^{\mu} w'_j} \quad \text{with} \quad w'_i = \ln(\mu + 0.5) - \ln i$$

for $i = 1, \dots, \mu$ and $w_{\mu+1} = \dots = w_\lambda = 0$.

- If μ best policies were selected uniformly at random from the λ candidates:

$$w_i \mathbf{y}_{i:\lambda} \sim \mathcal{N}(0, w_i^2 \mathbf{C})$$

$$\mathbf{y}_w = \sum_{i=1}^{\mu} w_i \mathbf{y}_{i:\lambda} \sim \mathcal{N}(0, \mu_w^{-1} \mathbf{C}) \quad \text{with} \quad \mu_w = \left(\sum_{i=1}^{\mu} w_i^2 \right)^{-1}$$

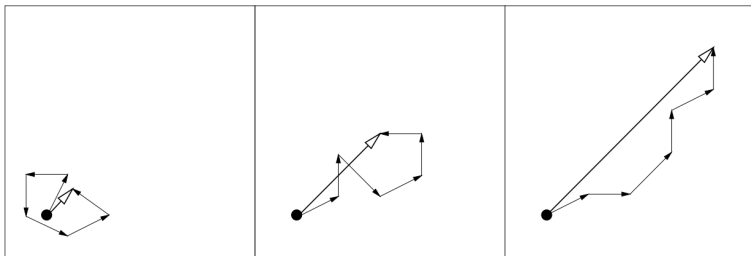
$$\sqrt{\mu_w} \mathbf{y}_w \sim \mathcal{N}(0, \mathbf{C})$$

$\Rightarrow$ we can treat $\sqrt{\mu_w} \mathbf{y}_w$ in the same way as the $\mathbf{y}_l$

Cumulative step size adaptation

Idea: Compare length of steps taken to the expected length under the assumption that the policies were ranked randomly.

If selection led to shorter or longer steps than expected without selection, we can adjust the step-size accordingly.



Conjugate evolution path

- Conjugate evolution path:

$$\mathbf{p}_\sigma \leftarrow (1 - c_\sigma)\mathbf{p}_\sigma + \sqrt{1 - (1 - c_\sigma)^2} \sqrt{\mu_w} \mathbf{C}^{-\frac{1}{2}} \mathbf{y}_w$$

- Assume $\mathbf{p}_\sigma \sim \mathcal{N}(0, \mathbf{I})$ on the RHS.
- Under the random selection, $\sqrt{\mu_w} \mathbf{y}_w \sim \mathcal{N}(0, \mathbf{C}) \Rightarrow \sqrt{\mu_w} \mathbf{C}^{-\frac{1}{2}} \mathbf{y}_w \sim \mathcal{N}(0, \mathbf{I})$.
Thus $\mathbf{p}_\sigma \sim \mathcal{N}(0, \mathbf{I})$ on the LHS.
- $c_c \in O(n)$ can be set to its default, faster timescale compared to learning $\mathbf{C}$.

Step-size update

Compare $\|\mathbf{p}_\sigma\|$ with the expected length under random selection $\mathbb{E}\|\mathcal{N}(0, \mathbf{I})\|$ and update:

$$\sigma \leftarrow \sigma \exp \left(\frac{c_\sigma}{d_\sigma} \left(\frac{\|\mathbf{p}_\sigma\|}{\mathbb{E}[\|\mathcal{N}(0, \mathbf{I})\|]} - 1 \right) \right)$$

If $\|\mathbf{p}_\sigma\| > \mathbb{E}\|\mathcal{N}(0, \mathbf{I})\| \rightarrow$ increase steps

If $\|\mathbf{p}_\sigma\| < \mathbb{E}\|\mathcal{N}(0, \mathbf{I})\| \rightarrow$ decrease steps

Damping factor $0 < \frac{c_\sigma}{d_\sigma} < 1$ can be set to default value.

Step-size update heuristic

One can think of the rule implementing the following heuristic:

- If the recent steps are correlated (i.e., go in the same direction), we have $\|\mathbf{p}_\sigma\| > \mathbb{E}\|\mathcal{N}(0, \mathbf{I})\|$.

Then it would be better to have used a larger step-size in hindsight to achieve the same change in fewer but larger steps. That is, the current step-size is too small and should be increased.

- If the recent steps are anti-correlated, we have $\|\mathbf{p}_\sigma\| < \mathbb{E}\|\mathcal{N}(0, \mathbf{I})\|$. This means that the algorithm jumped more back-and-forth than expected, i.e., oversteps (local) optima.

In this case, reducing the global step-size to make the search more local is a good strategy.

Basic CMA-ES algorithm

Algorithm: CMA-ES

Input: $\theta = \theta_{\text{init}}$, $\sigma = \sigma_{\text{init}}$, $\lambda \geq 5$ (default $4 + \lfloor 3 \ln n \rfloor$)

Set: $c_1 \approx 2/n^2$, $c_\mu \approx \mu_w/n^2$, $c_c \approx 4/n$, $c_\sigma \approx 1/\sqrt{n}$, $d_\sigma \approx 1$, $w_1 \geq w_2 \dots \geq w_\lambda \geq 0$
 with $\sum_{i=1}^\mu w_i = 1$, $\mu_w^{-1} = \sum_{i=1}^\mu w_i^2 \approx 3/\lambda$

1 Initialize $\mathbf{p}_\sigma = \mathbf{p}_c = \mathbf{0}$ and covariance matrix $\mathbf{C} = \mathbf{I}$ (unit matrix)

2 **repeat**

// sample new policies

3 **for** $l = 1, \dots, \lambda$ **do**

4 $\mathbf{y}_l \sim \mathcal{N}(\mathbf{0}, \mathbf{C})$

5 $\boldsymbol{\theta}_l \leftarrow \boldsymbol{\theta} + \sigma \mathbf{y}_l$

// evaluate new policies

6 **for** $l = 1, \dots, \lambda$ **do** evaluate $J^{\text{MC}}(\boldsymbol{\theta}_l)$

// update parameters

7 CMA-ES-update()

8 **until** *some stopping criterion is met*

Basic CMA-ES algorithm (cont.)

Procedure: CMA-ES-update()

// update mean

$$1 \quad \mathbf{y}_w = \sum_{i=1}^{\mu} w_i \boldsymbol{\theta}_{i:\lambda}$$

$$2 \quad \boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + c_m \sigma \mathbf{y}_w$$

// step size update

$$3 \quad \mathbf{p}_\sigma \leftarrow (1 - c_\sigma) \mathbf{p}_\sigma + \sqrt{1 - (1 - c_\sigma)^2} \sqrt{\mu_w} \mathbf{C}^{-\frac{1}{2}} \mathbf{y}_w$$

$$4 \quad \sigma \leftarrow \sigma \exp \left(\frac{c_\sigma}{d_\sigma} \left[\frac{\|\mathbf{p}_\sigma\|}{\mathbb{E}[\|\mathcal{N}(\mathbf{0}, \mathbf{I})\|]} - 1 \right] \right)$$

// covariance matrix update

$$5 \quad \mathbf{p}_c \leftarrow (1 - c_c) \mathbf{p}_c + \sqrt{1 - (1 - c_c)^2} \sqrt{\mu_w} \mathbf{y}_w$$

$$6 \quad \mathbf{C} \leftarrow \mathbf{C} + c_\mu \sum_{i=1}^{\lambda} w_i (\mathbf{y}_{i:\lambda} \mathbf{y}_{i:\lambda}^\top - \mathbf{C}) + c_1 (\mathbf{p}_c \mathbf{p}_c^\top - \mathbf{C})$$

CMA-ES summary

The CMA-ES

- represents “the state-of-the-art in evolutionary optimisation in real-valued $\mathbb{R}^n$ search spaces”,

Beyer: Evolution Strategies, Scholarpedia 2(8)

is “widely regarded as the state of the art in numerical optimization”,

Eiben, Smith: From evolutionary computation to the evolution of things, *Nature* 521, 2015

- is (“almost”) invariant under translation, rotation, flipping of search space and under order preserving transformations of objective function, and
- can be turned into a MO-CMA-ES for multi-objective optimisation.

Hansen, Ostermeier: Completely Derandomized Self-Adaptation in Evolution Strategies. *Evolut Comput*, 2001

Hansen et al.: Reducing the Time Complexity of the Derandomized Evolution Strategy with Covariance Matrix Adaptation (CMA-ES), *Evolut Comput*, 2003.

Auger, Hansen: A Restart CMA Evolution Strategy With Increasing Population Size, *CEC*, 2005

Igel, Hansen, Roth: Covariance Matrix Adaptation for Multi-objective Optimization, *volut Comput*, 2007

CMA-ES for reinforcement learning

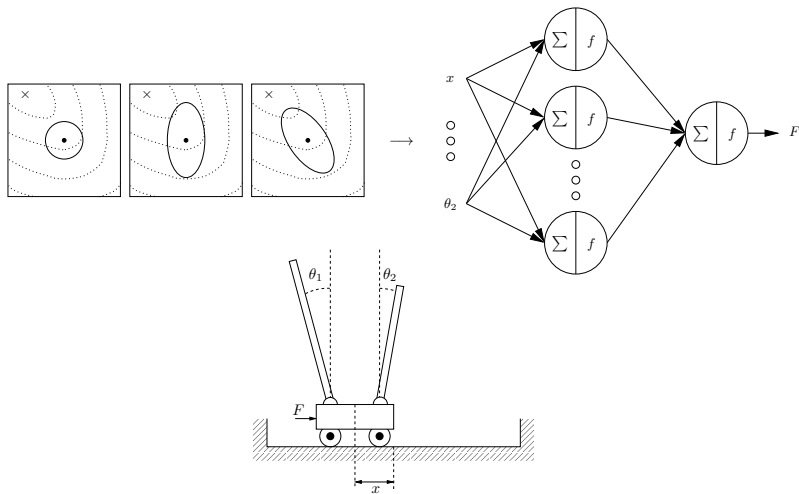
- **Idea:** Apply CMA-ES to reinforcement learning (RL, i.e., solving stochastic decision processes)!

Igel: Neuroevolution for Reinforcement Learning Using Evolution Strategies, *CEC*, 2003

- CMA-ES is used for direct policy search and searches a fixed parameterized class of functions.
- When ESs optimize neural networks we talk of **Neuroevolution Strategies (NeuroESs)**.

Heidrich-Meisner & Igel: Neuroevolution Strategies for Episodic Reinforcement Learning. *Journal of Algorithms* 64, 2009

Experimental setup



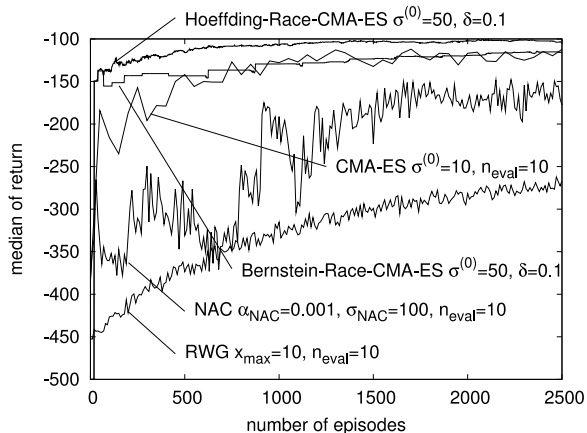
Empirical results

Several (independent) evaluations of the CMA-ES for RL give good results.

noisy Mountain-car

non-Markovian double pole balancing,
recurrent neural network policies

method	reward function	
	standard	damping
RWG	415,209	1,232,296
SANE	262,700	451,612
ESP	7,374	26,342
NEAT	–	6,929
RPG	(5,649)	–
CoSyNE	1,249	3,416
<i>CMA-ES</i>	860	1,141



The CMA-NeuroES ...

1. is a variable metric algorithm learning correlations between parameters,
2. extracts a search direction from the scalar reward signals,
3. is robust w.r.t. tuning of meta-parameters,
4. is based on ranking policies, which is robust w.r.t noise/uncertainty, and allows efficient uncertainty handling and distribution of roll-outs,

Heidrich-Meisner, Igel: Hoeffding and Bernstein Races for Selecting Policies in Evolutionary Direct Policy Search, *ICML*, 2009

5. can be restricted to diagonal covariance matrixes to avoid $O(n^2)$ scaling,
6. is applicable if the function approximators are non-differentiable,
7. performs great for episodic tasks with short episodes or in which the intermediate rewards do not carry (much) more information than their sum and the number of policy parameters is not too large.

Limitations of direct policy search

Direct policy search

- does not exploit intermediate rewards, just final Monte Carlo returns,
- has no concept of states, which would support credit assignment,
- in its canonical form is just defined for episodic tasks,
- is problematic if search space dimension gets too large.

Further reading

C. Igel. *Concepts of Deep Reinforcement Learning: A Short Course*, chapters 2 and 3, 2026

R. S. Sutton, A. G. Barto. *Reinforcement Learning: An Introduction*. 2nd edition, MIT Press, 2018

<http://incompleteideas.net/book/RLbook2020.pdf>

N. Hansen. *The CMA Evolution Strategy: A Tutorial*

<https://arxiv.org/abs/1604.00772>